CLIP

Guillermo Ruiz

En este notebook vamos a ver un ejemplo de como se puede usar CLIP para codificar imágenes y texto.

Carga de paquetes

```
#import drive
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/c
```

```
pip install open_clip_torch
```

```
Collecting open_clip_torch
  Downloading open_clip_torch-2.29.0-py3-none-any.whl.metadata (31 kB)
Requirement already satisfied: torch>=1.9.0 in /usr/local/lib/python3.10/dist-packages (from
Requirement already satisfied: torchvision in /usr/local/lib/python3.10/dist-packages (from c
Requirement already satisfied: regex in /usr/local/lib/python3.10/dist-packages (from open_cl
Collecting ftfy (from open_clip_torch)
  Downloading ftfy-6.3.1-py3-none-any.whl.metadata (7.3 kB)
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (from open_cli
Requirement already satisfied: huggingface-hub in /usr/local/lib/python3.10/dist-packages (fr
Requirement already satisfied: safetensors in /usr/local/lib/python3.10/dist-packages (from o
Requirement already satisfied: timm in /usr/local/lib/python3.10/dist-packages (from open_cli
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from torc
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.10/dist-pac
Requirement already satisfied: networkx in /usr/local/lib/python3.10/dist-packages (from torc
Requirement already satisfied: jinja2 in /usr/local/lib/python3.10/dist-packages (from torch)
Requirement already satisfied: fsspec in /usr/local/lib/python3.10/dist-packages (from torch)
```

```
Requirement already satisfied: sympy==1.13.1 in /usr/local/lib/python3.10/dist-packages (fro
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: wcwidth in /usr/local/lib/python3.10/dist-packages (from ftfy
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.10/dist-packages (f
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.10/dist-packages (from
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from hug
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from torchv
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.10/dist-packa
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10/dist-packages (f
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-pa
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages
Downloading open_clip_torch-2.29.0-py3-none-any.whl (1.5 MB)
                         1.5/1.5 MB 33.8 MB/s eta 0:00:00
Downloading ftfy-6.3.1-py3-none-any.whl (44 kB)
                         44.8/44.8 kB 4.2 MB/s eta 0:00:00
Installing collected packages: ftfy, open_clip_torch
Successfully installed ftfy-6.3.1 open_clip_torch-2.29.0
```

```python
import torch
from PIL import Image
import open_clip
import glob
```

Cargar el modelo CLIP de openCLIP

```python
model, _, preprocess = open_clip.create_model_and_transforms('ViT-B-32',
                        pretrained='laion2b_s34b_b79k')
model.eval()  # model in train mode by default, impacts some models with BatchNorm or stocha
tokenizer = open_clip.get_tokenizer('ViT-B-32')
```

```
/usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://hugg
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or
  warnings.warn(

open_clip_pytorch_model.bin:   0%|          | 0.00/605M [00:00<?, ?B/s]
```

Leer imágenes para probar

```
images_files = glob.glob("drive/MyDrive/clases/proces info 2025/Unidad 3/datos/images/*.jpg")
len(images_files)
```

```
200
```

Leer el archivo con las etiquetas de las imágenes

El archivo `miml_labels_1.csv` contiene una lista con los nombres de las imágenes y la categoría a la que pertenecen. Las posibles categorías son: Desierto, Montaña, Mar y Ocaso. Las primeras líneas del archivo son:

Filenames	desert	mountains	sea	sunset	trees
1.jpg	1	0	0	0	0
2.jpg	1	0	0	0	0
3.jpg	1	0	0	0	0
4.jpg	1	1	0	0	0
5.jpg	1	0	0	0	0

(La etiqueta trees no la vamos a usar)

Para leer el archivo usamos el siguiente código que deja los datos en el diccionario `data`.

```
data = {}
with open("drive/MyDrive/clases/proces info 2025/Unidad 3/datos/miml_labels_1.csv") as f:
  lines = f.readlines()
for l in lines[1:]:
  file_name, desert, mountains, sea, sunset, _ = l.split(",")
  data[file_name] = [desert, mountains, sea, sunset]
```

Crear los embedding para las etiquetas.

```
categorias = ["a desert", "a mountain", "a sea", "a sunset"]
text = tokenizer(categorias)
```

```
with torch.no_grad(), torch.cuda.amp.autocast():
    text_features = model.encode_text(text)
    text_features /= text_features.norm(dim=-1, keepdim=True)
```

```
FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autoca
  with torch.no_grad(), torch.cuda.amp.autocast():
```

Crear los embeddings para las imágenes.

Vamos a leer algunos archivos de imágenes y sacaremos sus embeddings para compararlos con los de las etiquetas.

```
img = Image.open(images_files[4])
```

```
image = preprocess(img).unsqueeze(0)
image.shape
```

```
torch.Size([1, 3, 224, 224])
```

```
with torch.no_grad(), torch.cuda.amp.autocast():
    image_features = model.encode_image(image)
    image_features /= image_features.norm(dim=-1, keepdim=True)
```

```
FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autoca
  with torch.no_grad(), torch.cuda.amp.autocast():
```